

컴퓨터 비전

Computer Vision

- 2D Image Processing 2-

동아대학교 소프트웨어대학 AI학과
2026년 1학기

임한신

RGB -> YUV 컬러 모델 변환 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/smarties.png'
urllib.request.urlretrieve(url, 'smarties.png')

# 이미지 읽기
img = cv2.imread('smarties.png')

# BGR에서 YUV로 컬러 공간 변환
yuv_img = cv2.cvtColor(img, cv2.COLOR_BGR2YUV)

# 채널별로 분리 (Y, U, V)
y_channel, u_channel, v_channel = cv2.split(yuv_img)
```

```
# 배열을 가로로 결합
combined_channels = np.hstack((y_channel, u_channel, v_channel))

print("--- [원본 이미지] ---")
cv2_imshow(img)

print("\n--- [YUV 채널 분리 결과: Y(밝기) | U(청색편차) | V(적색편차)] ---")
cv2_imshow(combined_channels)
```

히스토그램 평활화 실습

```
import urllib.request

# 샘플 영상 다운로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/vtest.avi'
urllib.request.urlretrieve(url, 'vtest.avi')

# 영상 읽기 설정
cap = cv2.VideoCapture('vtest.avi')

# 영상 정보 가져오기 (가로, 세로, FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # 저장 코덱 설정

# 3. 영상 쓰기(저장) 설정
out = cv2.VideoWriter('vtest_equalized.mp4', fourcc, fps, (width, height))
```

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

# YUV 변환 -> Y 평활화 -> 결합 -> BGR 복구
yuv = cv2.cvtColor(frame, cv2.COLOR_BGR2YUV)
y, u, v = cv2.split(yuv)

y_equalized = cv2.equalizeHist(y) # 평활화 적용

yuv_equalized = cv2.merge([y_equalized, u, v])
result_frame = cv2.cvtColor(yuv_equalized, cv2.COLOR_YUV2BGR)

# 처리된 프레임을 파일에 쓰기
out.write(result_frame)

cap.release()
out.release()

print("vtest_equalized.mp4")
```

Convolution (Filtering) 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg'
urllib.request.urlretrieve(url, 'baboon.jpg')
img = cv2.imread('baboon.jpg')

# 다양한 필터(커널) 정의

# 엠보싱(Embossing)
Emboss_kernel = np.array([[ -1, -1, 0],
                           [-1, 0, 1],
                           [ 0, 1, 1]])

# 샤프닝(Sharpening)
sharp_kernel = np.array([[ 0, -1, 0],
                           [-1, 5, -1],
                           [ 0, -1, 0]])
```

```
# 평균 (Average Blur): 5x5 크기로 평균
blur_kernel = np.ones((5, 5), np.float32) / 25

# 필터 적용 (cv2.filter2D)
emboss = cv2.filter2D(img, -1, emboss_kernel)
sharp = cv2.filter2D(img, -1, sharp_kernel)
blur = cv2.filter2D(img, -1, blur_kernel)

# 결과 출력
print("--- [1. 원 본] ---")
cv2_imshow(img)
print("\n--- [2. 엠보싱 필터] ---")
cv2_imshow(cv2.add(emboss, 128))
print("\n--- [3. 샤프닝 필터] ---")
cv2_imshow(sharp)
print("\n--- [4. 평균 필터] ---")
cv2_imshow(blur)
```

Convolution (Filtering) 실습 (Gaussian)

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg'
urllib.request.urlretrieve(url, 'baboon.jpg')
img = cv2.imread('baboon.jpg')

# 가우시안 필터 적용 (cv2.GaussianBlur)
# ksize: 커널 크기 (홀수), sigmaX: X방향 표준편차
blur_3x3 = cv2.GaussianBlur(img, (3, 3), 0)
blur_9x9 = cv2.GaussianBlur(img, (9, 9), 0)

# 직접 커널을 정의해서 적용하기
# 5x5 가우시안 커널 생성
kernel_5x5 = cv2.getGaussianKernel(5, 0)
# 1차원 커널을 외적하여 2차원 행렬로 변환
gaussian_kernel = np.outer(kernel_5x5, kernel_5x5)
custom_blur = cv2.filter2D(img, -1, gaussian_kernel)
```

```
# 결과 출력
print("--- [1. 원 본] ---")
cv2_imshow(img)
print("\n--- [2. 3x3 가우시안] ---")
cv2_imshow(blur_3x3)
print("\n--- [3. 9x9 가우시안] ---")
cv2_imshow(blur_9x9)
print("\n--- [4. 직접 정의한 5x5 커널] ---")
cv2_imshow(custom_blur)
```

Interpolation (보간) 실습

```
import urllib.request
from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/messi5.jpg'
urllib.request.urlretrieve(url, 'messi.jpg')
img = cv2.imread('messi.jpg')

# 테스트를 위해 이미지를 작게 축소 (100x100)
small_img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_AREA)

# 5배 확대 (500x500)
# (1) Nearest neighbor
nearest = cv2.resize(small_img, (500, 500), interpolation=cv2.INTER_NEAREST)
# (2) Bilinear
linear = cv2.resize(small_img, (500, 500), interpolation=cv2.INTER_LINEAR)
# (3) Bicubic
cubic = cv2.resize(small_img, (500, 500), interpolation=cv2.INTER_CUBIC)
```

```
# 결과 출력
print("--- [1. 축소된 원본 (100x100)] ---")
cv2_imshow(small_img)
print("\n--- [2. Nearest neighbor] ---")
cv2_imshow(nearest)
print("\n--- [3. Bilinear] ---")
cv2_imshow(linear)
print("\n--- [4. Bicubic] ---")
cv2_imshow(cubic)
```

감마 조절(gamma correction) 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

def adjust_gamma(image, gamma=1.0):
    # 0~255 범위의 룩업 테이블(LUT) 생성
    inv_gamma = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv_gamma) * 255
                      for i in np.arange(0, 256)]).astype("uint8")

    return cv2.LUT(image, table)

# 샘플 이미지 다운로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/fruits.jpg'
urllib.request.urlretrieve(url, 'fruits.jpg')
img = cv2.imread('fruits.jpg')

# 감마 값 적용
bright_gamma = adjust_gamma(img, gamma=2.2)
dark_gamma = adjust_gamma(img, gamma=0.4)

# 결과 출력 (가로 병합)
res = np.hstack((img, bright_gamma, dark_gamma))

print("--- [순서: 원본 | 감마 2.2 | 감마 0.4] ---")
cv2_imshow(res)
```

모폴로지(Morphology) 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/smarties.png'
urllib.request.urlretrieve(url, 'smarties.png')

img = cv2.imread('smarties.png')

# 흑백 이진화(Thresholding) 수행

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

# 구조 요소(Kernel) 생성 (5x5 크기)

kernel = np.ones((5, 5), np.uint8)

# 모폴로지 연산 적용

erosion = cv2.erode(binary, kernel, iterations=1) # 침식

dilation = cv2.dilate(binary, kernel, iterations=1) # 팽창
```

```
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel) # 열림

closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel) # 닫힘

# 결과 출력 (가로로 병합하여 비교)

res1 = np.hstack((binary, erosion, dilation))
res2 = np.hstack((binary, opening, closing))

print("--- [상단: 이진화 원본 | 침식 | 팽창] ---")
cv2_imshow(res1)

print("\n--- [하단: 이진화 원본 | 열림 | 닫힘] ---")
cv2_imshow(res2)
```


기하 변환 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드 (Sudoku)

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/sudoku.png'
urllib.request.urlretrieve(url, 'sudoku.png')

img = cv2.imread('sudoku.png')

rows, cols = img.shape[:2]

# 이동 변환(Translation) - 가로 100, 세로 50 이동
# 변환 행렬 M = [[1, 0, tx], [0, 1, ty]]
M_translation = np.float32([[1, 0, 100], [0, 1, 50]])
dst_translation = cv2.warpAffine(img, M_translation, (cols, rows))

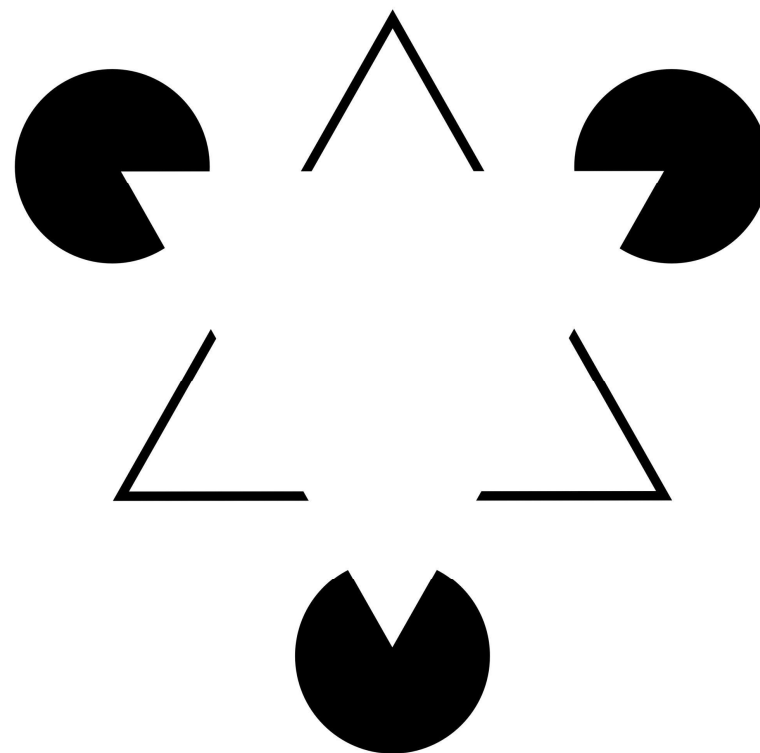
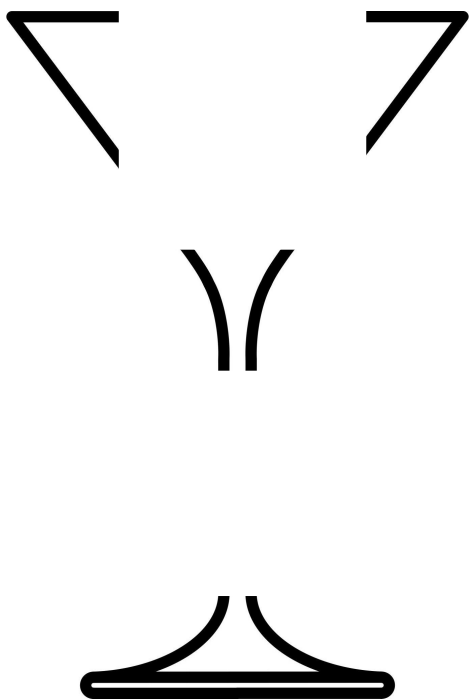
# 회전 + 크기 변환 (Rotation + Scale) - 중심축 기준 45도 회전, 0.7배 축소
# getRotationMatrix2D(중심좌표, 각도, 배율)
M_rotation = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 0.7)
dst_rotation = cv2.warpAffine(img, M_rotation, (cols, rows))
```

```
# 결과 출력
print("--- [1. 원본] ---")
cv2_imshow(img)

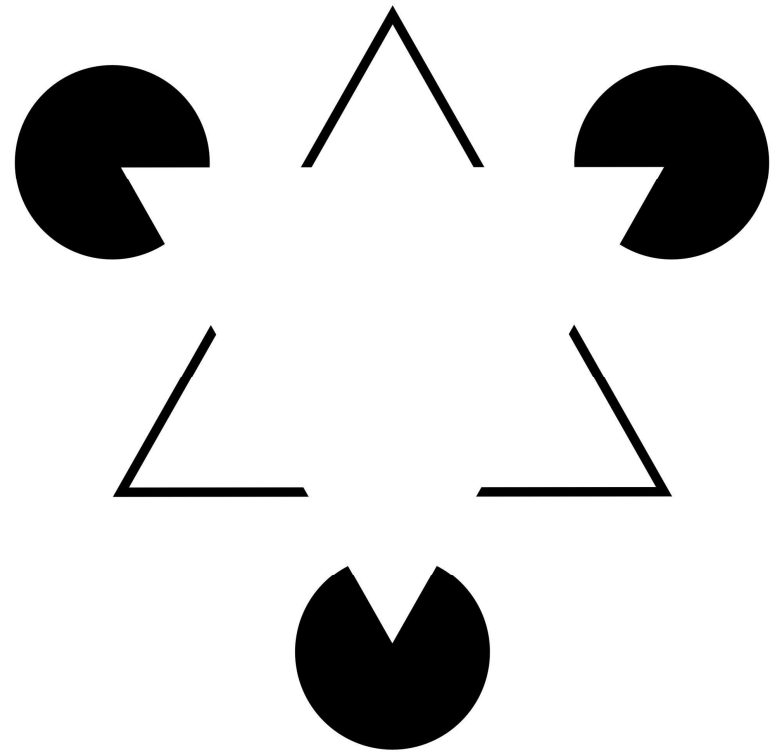
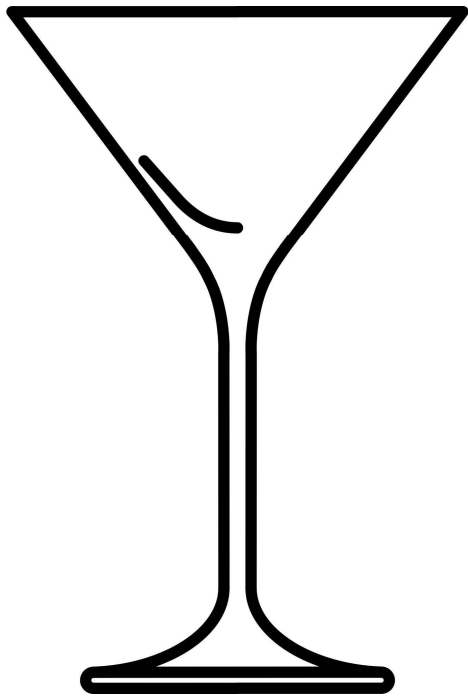
print("--- [2. 이동 변환 (Translation)] ---")
cv2_imshow(dst_translation)

print("\n--- [3. 회전 + 크기 변환 (Rotation + Scale)] ---")
cv2_imshow(dst_rotation)
```

에지 검출

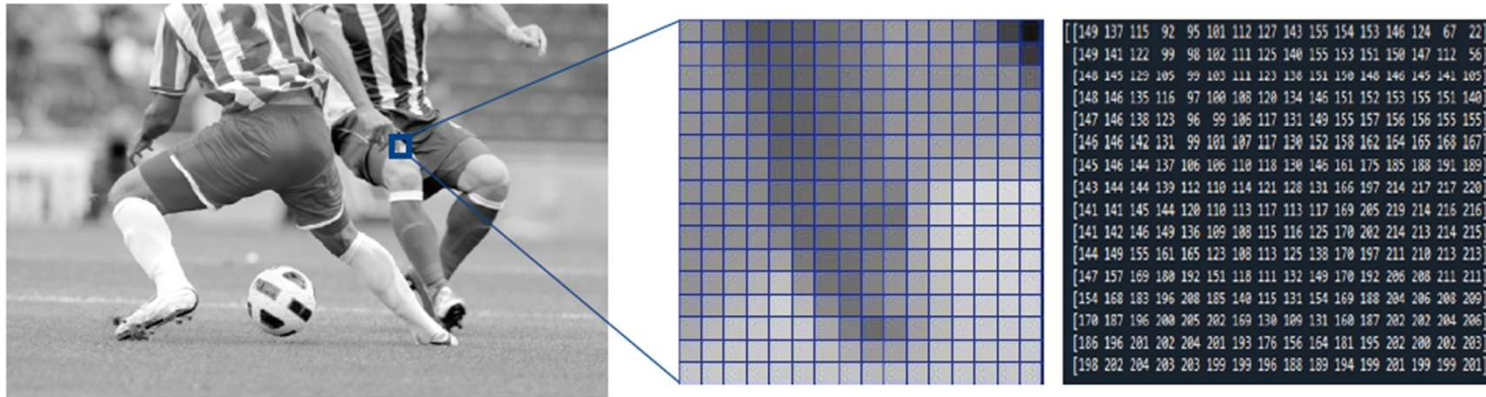


에지 검출



에지 검출

- 에지 검출 알고리즘
 - 물체 내부는 명암이 서서히 변하고 경계는 급격히 변하는 특성을 활용

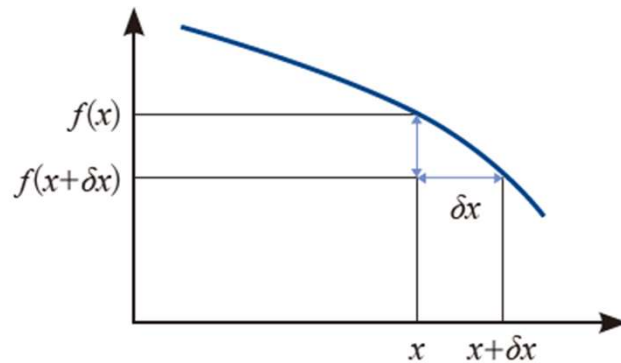


에지 검출

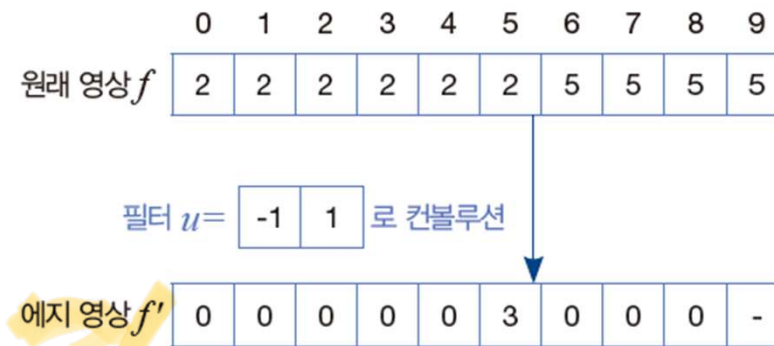
- 에지 검출 알고리즘
 - 미분을 디지털 영상에 적용하면

$$f'(x) = \frac{f(x+\delta x) - f(x)}{\delta x} = f(x+1) - f(x)$$

- 실제 구현은 필터 u 로 컨볼루션 (u 를 에지 연산자라 부름)



연속 함수의 미분



디지털 영상의 미분(필터 u 로 컨볼루션)

에지 검출

- 에지 검출 알고리즘
 - 1차 미분과 2차 미분

$$f''(x) = \frac{f'(x) - f'(x-\delta)}{\delta} = f'(x) - f'(x-1)$$

$$= (f(x+1) - f(x)) - (f(x) - f(x-1)) \quad \text{2차 미분}$$

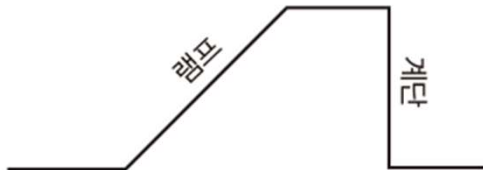
$$= f(x+1) - 2f(x) + f(x-1)$$

이 식을 구현하는 필터는

1	-2	1
---	----	---

두꺼운 에지로 인해 위치찾기 문제 발생

	0	1	2	3	4	5	6	7	8	9	0	1
원래 영상 f	3	3	3	3	5	7	7	7	7	3	3	3
필터 $u = \begin{bmatrix} -1 & 1 \end{bmatrix}$ 로 컨볼루션												
1차 미분 f'	0	0	0	2	2	0	0	0	-4	0	0	-
필터 $u = \begin{bmatrix} -1 & 1 \end{bmatrix}$ 로 컨볼루션												
2차 미분 f''	0	0	2	0	-2	0	0	-4	4	0	-	-



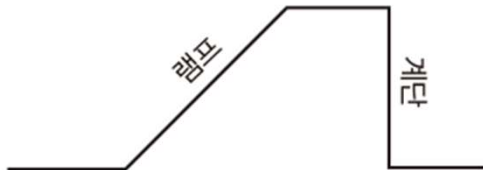
현실 세계에서 발생하는 램프 에지

에지 검출

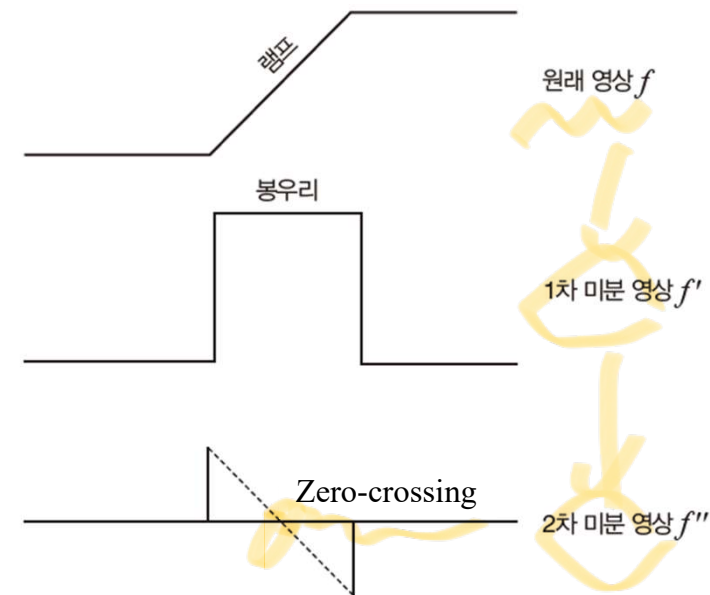
- 에지 검출 알고리즘
 - 1차 미분과 2차 미분

두꺼운 에지로 인해 위치찾기 문제 발생

	0	1	2	3	4	5	6	7	8	9	0	1
원래 영상 f	3	3	3	3	5	7	7	7	7	3	3	3
필터 $u = \begin{bmatrix} -1 & 1 \end{bmatrix}$ 로 컨볼루션												
1차 미분 f'	0	0	0	2	2	0	0	0	-4	0	0	-
필터 $u = \begin{bmatrix} -1 & 1 \end{bmatrix}$ 로 컨볼루션												
2차 미분 f''	0	0	2	0	-2	0	0	-4	4	0	-	-



현실 세계에서 발생하는 램프 에지



에지 검출

- 에지 검출 알고리즘

- 1차 미분에 기반한 에지 연산자
 - 크기가 2인 필터를 크기 3으로 확장

$$f'_x(y, x) = f(y, x+1) - f(y, x-1)$$

$$f'_y(y, x) = f(y+1, x) - f(y-1, x)$$

$u_x = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$ $u_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$

Handwritten notes: "↑ 1, ↑ 2, ↑ 3" and "가장" with arrows pointing to the 1s in the filters.

- 또한 1차원을 2차원으로 확장

$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$ $u_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

프레윗(Prewitt) 연산자

$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ $u_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

소벨(Sobel) 연산자

에지 검출

- 에지 검출 알고리즘

- 1차 미분에 기반한 에지 연산자

- 크기가 2인 필터를 크기 3으로 확장

$$f'_x(y, x) = f(y, x+1) - f(y, x-1)$$

$$f'_y(y, x) = f(y+1, x) - f(y-1, x)$$

$$u_x = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

- 또한 1차원을 2차원으로 확장

$$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

프레윗(Prewitt) 연산자

$$\text{에지 강도 : } (y, x) = \sqrt{f'_x(y, x)^2 + f'_y(y, x)^2}$$

$$\text{Gradient 방향 : } (y, x) = \arctan\left(\frac{f'_y(y, x)}{f'_x(y, x)}\right)$$

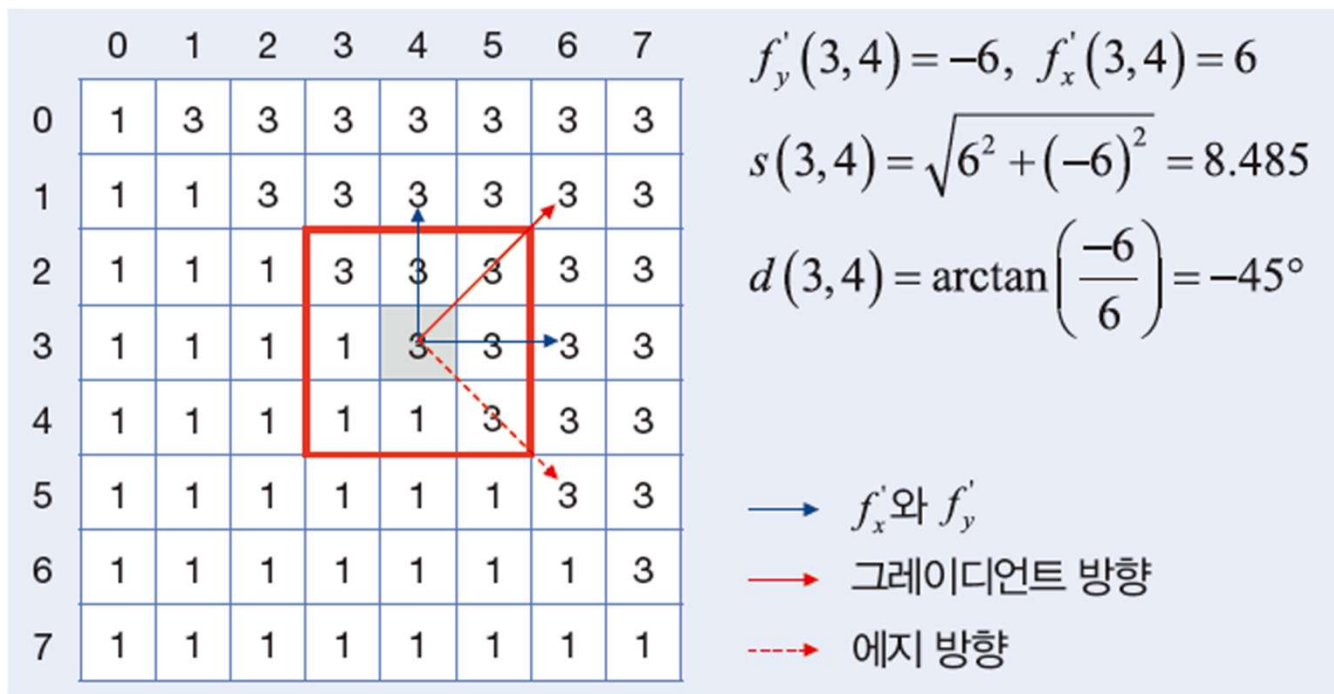
$$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

소벨(Sobel) 연산자

Sobel Gradient

에지 검출

- 에지 검출 알고리즘
 - 1차 미분에 기반한 에지 연산자



소벨 연산자 적용 예시

소벨(Sobel) 에지 검출 실습

```
import cv2
import numpy as np
import urllib.request
from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg'
urllib.request.urlretrieve(url, 'baboon.jpg')

# 이미지 읽기 및 흑백 변환
img = cv2.imread('baboon.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 소벨 에지 검출
# cv2.Sobel(영상, 출력 데이터 타입, x방향 미분값, y방향 미분값, 커널 크기)
# 데이터 타입은 정밀도를 위해 float64(CV_64F)를 사용한 뒤 나중에 8비트로 변환
합니다.
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3) # 수직선 검출
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3) # 수평선 검출
```

```
# 결과값의 절댓값을 취하고 8비트로 변환
abs_sobel_x = cv2.convertScaleAbs(sobel_x)
abs_sobel_y = cv2.convertScaleAbs(sobel_y)

# X축과 Y축 결과를 합쳐서 최종 에지 영상 생성
sobel_combined = cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y, 0.5, 0)

# 결과 출력
print("--- [원본 이미지] ---")
cv2_imshow(img)
print("\n--- [Sobel X (수직 경계)] ---")
cv2_imshow(abs_sobel_x)
print("\n--- [Sobel Y (수평 경계)] ---")
cv2_imshow(abs_sobel_y)
print("\n--- [Sobel Combined (최종 에지)] ---")
cv2_imshow(sobel_combined)
```

에지 검출

- 캐니(Canny) 에지 검출 알고리즘
 - 기존 방식들의 단점
 - 노이즈에 취약하거나 에지가 너무 두껍게 표현됨
 - 단계
 1. 가우시안 필터를 적용하여 노이즈 제거
 2. 소벨(Sobel) 마스크 등을 사용해 각 픽셀에서의 밝기 변화량(크기)과 변화 방향(각도)을 구함
 3. 에지가 두껍게 나타나는 것을 방지하기 위해 그래디언트 방향에서 가장 값이 큰(극대점인) 픽셀만 남기고 나머지는 제거
 4. 이중 임계값(T_{high} , T_{low})으로 강한 에지(T_{high} 보다 큼)와 약한 에지(T_{high} 와 T_{low} 의 사이)를 분류
 5. 약한 에지 중에서 강한 에지와 연결된 것만 진짜 에지로 인정하고 나머지는 제거



캐니(Canny) 에지 검출 실습

```
import cv2
import numpy as np
import urllib.request
from google.colab.patches import cv2_imshow

url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/fruits.jpg'
urllib.request.urlretrieve(url, 'fruits.jpg')

# 이미지 읽기
img = cv2.imread('fruits.jpg')

# 에지 검출 성능을 높이기 위해 흑백(Grayscale) 변환
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 가우시안 블러로 노이즈 제거
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# 캐니 에지 적용
# cv2.Canny(이미지, 하한 임계값, 상한 임계값)
edges = cv2.Canny(blur, 100, 200)

# 결과 출력 (원본과 에지 영상)
print("--- 원본 이미지 ---")
cv2_imshow(img)
print("\n--- 캐니 에지 결과 ---")
cv2_imshow(edges)
```

Summary
Gaussian + Gradient